

Anomaly Detection 🕵️

This is an unsupervised learning algorithm that examines an unlabeled dataset of "normal" events and learns to detect or "flag" unusual or anomalous events.

The Problem

Imagine you are a manufacturer of aircraft engines.

- You produce many engines, and you want to ensure they are reliable.
- You can measure features for each engine, such as:
 - x_1 : Heat generated
 - x_2 : Vibration intensity
- **Dataset:** You have a dataset of m engines that you have manufactured. Most of these are normal, good engines.
- **Goal:** When a new engine (x_{test}) is made, you want to know: "Is this engine similar to the ones I've seen before (normal), or is it strange (anomalous)?"

How it Works: Density Estimation

The core technique used for anomaly detection is called **Density Estimation**.

1. Build a Model ($p(x)$):

- The algorithm uses the training data to build a probability model $p(x)$.
- This model tells you the probability of seeing a specific set of features.
- Values of x that are "normal" (seen often) will have a **high probability**.
- Values of x that are "unusual" (rare) will have a **low probability**.

2. Set a Threshold (ϵ):

- You choose a small number ϵ (epsilon) as your cutoff threshold.

3. Detect Anomalies:

- For a new example x_{test} , calculate $p(x_{test})$.
- **If $p(x_{test}) < \epsilon$:** The probability is very low. Flag it as an **anomaly**.
- **If $p(x_{test}) \geq \epsilon$:** The probability is high enough. It is **normal**.

Real-World Applications

Anomaly detection is extremely common in industry, used for ensuring quality and security.

- **Fraud Detection:**

- **Context:** Websites tracking user behavior (login frequency, typing speed, transaction volume).
- **Usage:** Model $p(x)$ for a user's typical behavior. If a user's activity suddenly has a very low probability (e.g., logging in from a new country and typing much faster), flag it for security review (e.g., ask for 2-factor authentication).
- **Financial Fraud:** Identifying unusual credit card transactions.
- **Manufacturing (Quality Control):**
 - **Context:** Producing items like aircraft engines, smartphones, or circuit boards.
 - **Usage:** Measure features of every unit. If a unit has anomalous measurements (e.g., high vibration), inspect it for defects before shipping.
- **Monitoring Data Centers:**
 - **Context:** Managing thousands of computer servers.
 - **Usage:** Monitor features like memory usage, CPU load, and network traffic. If a machine behaves anomalously (e.g., CPU load spikes while network traffic is zero), it might be broken, overheating, or hacked.

The Gaussian (Normal) Distribution 🔔

To perform anomaly detection, we need a way to calculate the probability $p(x)$ that a specific data point belongs to a dataset. We do this using the **Gaussian Distribution**, also known as the **Normal Distribution** or the **Bell Curve**.

Key Parameters

The shape of the Gaussian distribution is determined by two parameters:

- 1. **μ (Mu) - The Mean:**
 - This describes the **center** of the distribution.
 - It determines where the peak of the bell curve sits.
- 2. **σ (Sigma) - The Standard Deviation:**
 - This describes the **width** of the distribution.
 - It determines how spread out the data points are.
 - σ^2 (**Sigma squared**) is called the **Variance**.

Visualizing the Distribution

The area under the curve always sums to **1** (representing total probability). Therefore, changing the width (σ) forces the height to change as well.

Scenario	Shape of Curve	Interpretation
Standard ($\mu = 0, \sigma = 1$)	Standard Bell Shape	Centered at 0, standard spread.
Small σ (e.g., $\sigma = 0.5$)	Tall and Skinny	Data is tightly clustered around the mean.
Large σ (e.g., $\sigma = 2$)	Short and Wide	Data is very spread out (high variance).
Change μ	Shifts Left/Right	The shape stays the same, but the center moves.

The Mathematical Formula

The probability of x , denoted as $p(x)$, is calculated using the following formula:

$$p(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

- This formula produces the bell-shaped curve.
 - Points near the mean μ will produce a high value for $p(x)$.
 - Points far from the mean (in the "tails") will produce a very low value for $p(x)$.
-

Parameter Estimation: Learning from Data

If you have a dataset of m examples $(x^{(1)}, \dots, x^{(m)})$, how do you find the correct μ and σ to fit your data?

You calculate the **Maximum Likelihood Estimates**:

1. Estimate the Mean (μ):

Simply calculate the average of your training examples.

$$\mu = \frac{1}{m} \sum_{i=1}^m x^{(i)}$$

2. Estimate the Variance (σ^2):

Calculate the average of the squared differences between each example and the mean.

$$\sigma^2 = \frac{1}{m} \sum_{i=1}^m (x^{(i)} - \mu)^2$$

***Note on Statistics:** Some statistics courses recommend dividing by $m - 1$ instead of m for the variance. In machine learning (and for large datasets), the difference is negligible. Using $1/m$ is standard practice here.*

Applying to Anomaly Detection

Once you have calculated μ and σ^2 for your data:

- **Normal examples:** Will fall near the center (μ). $p(x)$ will be **high**.
- **Anomalous examples:** Will fall far away in the tails. $p(x)$ will be **low**.

Currently, we have looked at x as a single number (one feature). However, real-world problems usually involve multiple features $(x_1, x_2, \dots, x_n)$.

The Anomaly Detection Algorithm 🔍

We can now build the full algorithm using the concept of the Gaussian distribution.

The Model: Multiplying Probabilities

We have a training set where each example x has n features $(x_1, \dots, x_n)$.

- *Example:* Aircraft engine with 2 features (Heat, Vibration). $n = 2$.

To estimate the total probability $p(x)$ of an entire vector of features, we assume the features are statistically independent (though the algorithm works well even if they aren't) and **multiply the probabilities of each individual feature**.

$$p(x) = p(x_1; \mu_1, \sigma_1^2) \times p(x_2; \mu_2, \sigma_2^2) \times \dots \times p(x_n; \mu_n, \sigma_n^2)$$

This can be written compactly as:

$$p(x) = \prod_{j=1}^n p(x_j; \mu_j, \sigma_j^2)$$

- $\prod$ is the product symbol (like $\sum$ for summation), meaning "multiply all these terms together."
 - Each feature j has its own mean μ_j and variance σ_j^2 .
-

The Algorithm Steps

1. Choose Features

Select features x_i that you believe are indicative of anomalous behavior.

2. Fit Parameters (μ and σ^2)

Calculate the mean and variance for **each feature j** using the training set.

- $\mu_j = \frac{1}{m} \sum_{i=1}^m x_j^{(i)}$ (Average of feature j)
- $\sigma_j^2 = \frac{1}{m} \sum_{i=1}^m (x_j^{(i)} - \mu_j)^2$ (Variance of feature j)

3. Compute $p(x)$ for New Examples

Given a new example x (e.g., a new engine), calculate its total probability:

$$p(x) = \prod_{j=1}^n \frac{1}{\sqrt{2\pi\sigma_j^2}} \exp\left(-\frac{(x_j - \mu_j)^2}{2\sigma_j^2}\right)$$

4. Evaluation (Flag Anomaly)

Compare the calculated $p(x)$ to a threshold ϵ .

- If $p(x) < \epsilon$: Flag as **ANOMALY**.
 - If $p(x) \geq \epsilon$: Consider it **NORMAL**.
-

Intuition

- The algorithm fits a bell curve to *each* feature independently.
 - If a new example has even **one** feature that is very far from the mean (e.g., excessively high heat), the probability $p(x_j)$ for that specific feature will be tiny.
 - Because we are multiplying, **multiplying by a tiny number makes the total product tiny**.
 - Therefore, if any single feature is anomalous, the total $p(x)$ will drop below ϵ , and the system will flag it.
-

Example Visualization

- Imagine data with 2 features.
 - x_1 : Heat (Mean=5, Sigma=2)
 - x_2 : Vibration (Mean=3, Sigma=1)
- Multiplying these two distributions creates a 3D "hill" of probability.
 - The peak (highest probability) is in the center, where the means are.
 - The edges (low probability) are where anomalies live.
- **Test 1 (Center):** $p(x) = 0.4$ (Normal)
- **Test 2 (Far Edge):** $p(x) = 0.0021$ (Anomaly)

Developing and Evaluating an Anomaly Detection System

Building an effective machine learning system involves making many decisions (choosing features, setting parameters like ϵ). The process is significantly faster and more effective if you have a way to **evaluate** your system with a concrete number (a "real number evaluation").

Labeled vs. Unlabeled Data

Although anomaly detection is fundamentally an **unsupervised** learning technique (learning from normal data), evaluating it is much easier if we assume we have a **small amount of labeled data**.

- **$y = 0$ (Normal):** The vast majority of your data (e.g., 10,000 good engines).
- **$y = 1$ (Anomalous):** A small handful of known anomalies (e.g., 20 flawed engines you've seen in the past).

Splitting the Data

We split our data into three sets, ensuring that the **Training Set** consists only of "normal" examples (or examples we *assume* are normal), while the **Cross Validation (CV)** and **Test** sets contain a mix of normal examples and the few known anomalies.

Example Scenario:

- **Total Data:** 10,000 Good Engines ($y = 0$), 20 Anomalous Engines ($y = 1$).

Dataset Split	Composition	Purpose
Training Set	6,000 Good Engines ($y = 0$)	Used to fit the Gaussian model ($p(x)$). The algorithm learns what "normal" looks like.
CV Set	2,000 Good Engines ($y = 0$) 10 Anomalous Engines ($y = 1$)	Used to tune parameters (like ϵ) and select features.
Test Set	2,000 Good Engines ($y = 0$) 10 Anomalous Engines ($y = 1$)	Used for the final evaluation of the system's performance.

(Note: If you have very few anomalies—e.g., only 2—you might skip the Test Set and put all anomalies into the CV Set. This risks overfitting but is sometimes necessary.)

The Evaluation Process

1. **Train:** Fit the model $p(x)$ using the **Training Set**.
2. **Tune:** Use the **CV Set** to make predictions:
 - Calculate $p(x)$ for every example in the CV set.
 - Predict $y = 1$ if $p(x) < \epsilon$, and $y = 0$ otherwise.
 - **Adjust ϵ :** If the model misses the anomalies, increase ϵ . If it flags too many normal engines, decrease ϵ .
3. **Evaluate:** Once you are happy with the performance on the CV set, run the final model on the **Test Set** to get an unbiased estimate of its accuracy.

Metrics for Skewed Data

Because anomalies are rare (the data is highly "skewed"), standard classification accuracy is often a bad metric. (A model that always predicts "Normal" would have 99.9% accuracy but be useless).

Instead, use metrics like:

- **True Positive / False Positive / False Negative / True Negative** counts.
- **Precision and Recall.**
- **F1-Score.**

These metrics will tell you if your algorithm is actually finding the rare anomalies without falsely flagging too many normal examples.

Anomaly Detection vs. Supervised Learning

When you have a dataset with very few positive examples ($y = 1$) and a large number of negative examples ($y = 0$), the choice between using Anomaly Detection or Supervised Learning can be subtle.

Here is the framework for deciding which to use.

1. Based on Number of Examples

The most immediate indicator is the volume of positive data available.

Anomaly Detection	Supervised Learning
Very small number of positive examples ($y = 1$). (e.g., 0 to 20 examples).	Large number of positive examples ($y = 1$).
Large number of negative examples ($y = 0$).	Large number of negative examples ($y = 0$).
The algorithm learns $p(x)$ using only the negative examples. The few positive examples are reserved for the Cross-Validation and Test sets.	The algorithm uses both positive and negative examples during training to learn the boundary between classes.

2. Based on the Nature of the "Anomaly"

The more important distinction lies in whether future anomalies will look like past anomalies.

Use Anomaly Detection When:

- There are many **different types** of anomalies.
- It is hard for an algorithm to learn what an anomaly "looks like" from a small set of positive examples.
- **Crucial:** Future anomalies may look **nothing like** the anomalies you have seen so far.
- **Method:** The algorithm builds a model of "Normal." Anything that deviates from "Normal" is flagged.

Use Supervised Learning When:

- There are enough positive examples for the algorithm to get a sense of what they look like.
- **Crucial:** Future positive examples are likely to be **similar** to the positive examples in the training set.

- **Method:** The algorithm learns a boundary that specifically separates positive examples from negative examples.

Real-World Examples

Application	Recommended Algorithm	Reason
Financial Fraud	Anomaly Detection	Hackers constantly invent new methods of fraud. A new attack might look completely different from a previous one.
Email Spam	Supervised Learning	Spam often follows predictable patterns (selling drugs, phishing links). Future spam usually resembles past spam.
Manufacturing (Unknown Defects)	Anomaly Detection	Example: Aircraft engines. A failure might happen for a brand new reason (a new type of crack or material failure) never seen before.
Manufacturing (Known Defects)	Supervised Learning	Example: Scratched smartphone screens. You have seen thousands of scratches; you just want to find <i>more of the same</i> defect.
Data Center Security	Anomaly Detection	Hackers find new exploits (Zero-day attacks). The machine behaves in a "weird" way that hasn't been defined yet.
Weather / Medical	Supervised Learning	Diagnosing a specific disease or predicting rain vs. sun. The "positive" cases (having the disease/rain) are well-defined and repetitive.

Summary

- **Anomaly Detection:** "I don't know what might go wrong, but I know what 'normal' looks like. Alert me if *anything* weird happens."
- **Supervised Learning:** "I know exactly what goes wrong (or what I'm looking for). Here are 1,000 examples of it. Find me more of *this specific thing*."

Choosing Features for Anomaly Detection

In Supervised Learning, the algorithm has labels (y) to guide it, so it can often learn to ignore irrelevant features on its own. However, in **Anomaly Detection**, the algorithm learns purely from unlabeled data. Therefore, **choosing the right features is critical** to the system's success.

1. Transforming Non-Gaussian Features

The anomaly detection algorithm assumes that your features follow a **Gaussian (Normal) Distribution** (the bell curve). If your data is "skewed" (not bell-shaped), the algorithm might perform poorly.

The Process:

1. **Plot a Histogram:** Use code (like `plt.hist`) to visualize your feature.
2. **Check Shape:** Does it look like a bell curve?
 - **If Yes:** Good to go.
 - **If No:** Apply a transformation to shape it into a bell curve.

Common Transformations:

If a feature x is skewed (e.g., has a long tail to the right), try these transformations to "squash" the tail:

- **Log:** $\log(x)$ (Note: if x can be 0, use $\log(x + c)$ where c is a small constant).
- **Roots:** $\sqrt{x}$ (or $x^{0.5}$), $x^{1/3}$, etc.
- **Exponentiation:** $x^{0.4}$, etc.

You can simply try different transformations in a notebook until the histogram looks roughly Gaussian. **Important:** Whatever transformation you apply to the training set, you must also apply to the Cross-Validation and Test sets.

2. Error Analysis for Anomaly Detection

If your model is not performing well on your Cross-Validation set, performing **Error Analysis** can help you find better features.

Scenario:

Your model fails to flag a specific anomalous example (x_{anom}) because it assigns it a high probability ($p(x) \geq \epsilon$).

The Solution:

1. **Inspect the Example:** Look closely at that specific x_{anom} .
 2. **Ask:** "Why is this anomalous? What is strange about it compared to the normal examples?"
 3. **Create a New Feature:** Try to design a new feature that takes on an unusually large or small value *only* for this anomaly.
- **Example:**
 - You are detecting fraud.
 - The model misses a fraudster because their "Transaction Count" (x_1) is normal.
 - **Inspection:** You notice this user types insanely fast.
 - **Action:** Add a new feature: "Typing Speed" (x_2). Now the anomaly stands out.
-

3. Combining Features (Feature Engineering)

Sometimes, an anomaly isn't obvious when looking at features individually, but becomes obvious when looking at the **relationship** between features.

Example: Data Center Monitoring

Imagine you are monitoring a computer server.

- x_1 : CPU Load
- x_2 : Network Traffic

The Anomaly: A machine is stuck in an infinite loop.

- It has **High CPU Load** (which is normal for a busy server).
- It has **Low Network Traffic** (which is normal for an idle server).
- However, the *combination* (High CPU with Low Traffic) is very strange.

The Fix: Create a new feature to capture this relationship.

- $x_{new} = \frac{\text{CPU Load}}{\text{Network Traffic}}$
 - For the anomalous machine, this ratio will be incredibly large, allowing the algorithm to flag it easily.
-

Summary of Course 3, Week 1

Congratulations on finishing the first week! You have now mastered two major Unsupervised Learning techniques:

1. **Clustering (K-Means):** For grouping data.
2. **Anomaly Detection:** For finding unusual events.